BARRETT HODGSON PAKISTAN

Managed File Transfer Solution

TECHNICAL DOCUMENTATION

For New Developers & IT Administrators

Version 1.0 | July 2026 | Barrett Hodgson IT Services

TABLE OF CONTENTS

1. PROJECT OVERVIEW

Project Name: Barrett Hodgson Managed File Transfer (MFT) Solution

Purpose: Secure, on-premise file sharing platform with Active Directory integration for enterprise users.

Technology Stack:

Layer	Technology	Version
Backend Framework	Python Flask	3.0.0
HTTP Server	Werkzeug (Development)	3.0.1
LDAP/AD	ldap3	2.9.1
Database	SQLite3	Built-in
Frontend	HTML5 + CSS3 + Vanilla JS	N/A
Email	PowerShell Send-MailMessage	Windows

[INFO] This is a Python-based web application. No JavaScript frameworks (React, Vue, Angular) are used. All frontend is pure HTML/CSS/JS.

2. SYSTEM ARCHITECTURE

2.1 High-Level Architecture

The application follows a simple 3-tier architecture:

- Presentation Layer: HTML templates (Jinja2) rendered by Flask
- Application Layer: Python Flask routes, business logic, AD authentication
- Data Layer: SQLite database + local filesystem for file storage

2.2 Data Flow Diagram

```
[User Browser] → [Flask App (Port 5000)] → [Active Directory (Port 389/636)]
                        ↓
              [SQLite Database]
                        ↓
              [Local Filesystem (uploads/)]
                        ↓
              [Exchange Server (Port 25)] → [Recipient Email]
```

2.3 Network Diagram

```
Internet User (External)
    ↓
Sophos XG310 Firewall (Port Forward 5000)
    ↓
Local PC (172.16.0.51:5000) ← Flask App
    ↓
    ├── AD Server (172.16.0.12:389)
    ├── Exchange Server (172.16.0.17:25)
    └── SQLite DB (audit_logs.db)
```

[WARNING] For external access, Sophos XG310 must forward port 5000 (or 80/443) to the internal PC IP.

3. FILE STRUCTURE & ORGANIZATION

3.1 Directory Tree

```
FILE-SHARE-APP/
├── app.py                  ← Main Flask application (ALL logic)
├── app copy.py             ← Backup copy
├── audit_logs.db           ← SQLite database (auto-created)
├── requirements.txt        ← Python dependencies
├── check_ad_groups.py      ← AD group verification utility
├── static/                 ← Static assets (images, CSS, JS)
│   └── Logo.png            ← Company logo
├── templates/              ← Jinja2 HTML templates
│   ├── login.html          ← AD login page
│   ├── index.html          ← Main upload page
│   ├── result.html         ← Upload success page
│   ├── admin_logs.html     ← Admin audit dashboard
│   ├── index_copy.html     ← Backup
│   └── login_copy.html     ← Backup
├── uploads/                ← Uploaded files storage
│   └── [UUID].pdf          ← Example: a1b2c3d4e5f6.pdf
└── temp_uploads/           ← Temporary upload buffer
```

3.2 Key Files Explained

File	Purpose
app.py	Main application. Contains ALL routes, AD functions, email logic, database setup.
templates/*.html	Jinja2 templates. Flask renders these with {{ variables }} passed from Python.
uploads/	Physical file storage. Files saved with UUID names (e.g., a1b2c3d4.pdf).
audit_logs.db	SQLite database. Stores upload records, download counts, email status.
requirements.txt	Python package list. Install with: pip install -r requirements.txt

4. DATABASE SCHEMA

4.1 Table: uploads

Database: SQLite (audit_logs.db)

Column	Type	Description	Example
id	INTEGER PK	Auto-increment primary key	1
file_id	TEXT UNIQUE	12-char UUID for public access	a1b2c3d4e5f6
filename	TEXT	Stored filename (UUID + ext)	a1b2c3d4e5f6.pdf
original_name	TEXT	Original user filename	report.pdf
file_size	INTEGER	File size in bytes	5242880
file_path	TEXT	Absolute path to file	uploads/a1b2c3d4e5f6.pdf
download_link	TEXT	Public download URL	http://.../download/abc123
uploaded_by	TEXT	Display name of uploader	Habib Ur Rehman
uploaded_by_domain	TEXT	AD domain	barretthodgson.com
recipient_email	TEXT	Email of recipient	user@domain.com
upload_time	TEXT	Upload timestamp	2026-07-13 14:30:00
expiry_time	TEXT	Auto-delete timestamp	2026-07-16 14:30:00
ip_address	TEXT	Uploader IP address	172.16.40.100
user_agent	TEXT	Browser info	Mozilla/5.0...
status	TEXT	Upload status	SUCCESS
email_sent	BOOLEAN	Email delivery status	1 (True)
email_error	TEXT	Error message if failed	SMTP timeout
download_count	INTEGER	Times downloaded	3
last_download_time	TEXT	Last download timestamp	2026-07-14 09:00:00
is_external_allowed	BOOLEAN	Sender has external rights	1 (True)
recipient_is_external	BOOLEAN	Recipient outside domain	1 (True)
auth_method_used	TEXT	Email auth method	PowerShell
message	TEXT	User message to recipient	Please review ASAP

[INFO] SQLite is file-based. No separate database server needed. The .db file is created automatically on first run.

5. CONFIGURATION GUIDE

5.1 Critical Variables in app.py

Variable	Location	Description
AD_SERVER	Line 22	IP address of Active Directory server
AD_PORT	Line 23	LDAP port (389 for plain, 636 for SSL)
AD_DOMAIN	Line 25	Your domain name (e.g., barretthodgson.com)
AD_BASE_DN	Line 26	LDAP search base (DC=barretthodgson,DC=com)
AD_SERVICE_USER	Line 29	Read-only AD service account username
AD_SERVICE_PASSWORD	Line 30	Service account password (SECURITY RISK - see below)
EXCHANGE_SERVER	Line 33	Exchange SMTP server IP
EXCHANGE_PORT	Line 34	SMTP port (25 for plain, 587 for TLS)
BASE_URL	Line 38	Public-facing URL for download links

5.2 Example Configuration

```
# ========== AD / DOMAIN CONFIGURATION ==========
AD_SERVER = "172.16.0.12"          # Your AD server IP
AD_PORT = 389                       # LDAP port
AD_SSL_PORT = 636                   # LDAPS (SSL) port
AD_DOMAIN = "barretthodgson.com"   # Your domain
AD_BASE_DN = "DC=barretthodgson,DC=com"

# Service Account for AD Read
AD_SERVICE_USER = "itservices"    # Read-only AD account
AD_SERVICE_PASSWORD = "Dont_Enter"  # ⚠ CHANGE THIS! Use env variable

# Exchange Settings
EXCHANGE_SERVER = "172.16.0.17"    # Exchange server IP
EXCHANGE_PORT = 25                  # SMTP port

# Application Settings
BASE_URL = "http://172.16.40.68:5000"  # ← CHANGE TO PUBLIC IP FOR EXTERNAL ACCESS
UPLOAD_FOLDER = 'uploads'           # File storage folder
FILE_EXPIRY_DAYS = 3                # Default expiry (1-14 days)
FILE_EXPIRY_MAX = 14                # Maximum allowed expiry
```

[WARNING] For external access, change BASE_URL to your public static IP or domain. Example: BASE_URL = 'https://files.barretthodgson.com'

6. AUTHENTICATION FLOW (ACTIVE DIRECTORY)

6.1 How Login Works

- User enters username/password on login.html
- Flask receives POST request to /login route
- verify_ad_credentials() function tries 5 authentication formats:
  - a) UPN: username@barretthodgson.com
  - b) NTLM: barretthodgson.com\username
  - c) Plain: username
  - d) SSL UPN: username@barretthodgson.com (port 636)
  - e) SSL NTLM: barretthodgson.com\username (port 636)
- If ANY format succeeds → user is authenticated
- Session stored: session['ad_username'] = username
- User redirected to index.html (upload page)

6.2 User Detection Priority

```
Flask checks user identity in this order:
1. session['ad_username']          ← Logged in via /login page
2. request.environ['REMOTE_USER']  ← Windows Integrated Auth (IIS)
3. request.headers['X-Remote-User']  ← Reverse proxy header
4. getpass.getuser()                 ← Local system user (localhost only)
```

[INFO] For production IIS deployment, Windows Integrated Auth (Kerberos/NTLM) can be used instead of manual login.

7. FILE UPLOAD & DOWNLOAD MECHANISM

7.1 Upload Process

- User selects file via drag-and-drop or click (index.html)
- JavaScript validates file size (client-side, max 2GB)
- AJAX POST request sends file to /upload route
- Flask receives file via request.files['file']
- UUID generated: file_id = str(uuid.uuid4())[:12]
- File saved as: uploads/{file_id}.{original_extension}
- Database record created with metadata
- Email sent to recipient via PowerShell
- Sender acknowledgment email sent
- Result page displayed with download link

7.2 File Naming Convention

```
Original File:    "Annual_Report_2026.pdf"
                  ↓
UUID Generated:   "a1b2c3d4e5f6"
                  ↓
Stored As:        "uploads/a1b2c3d4e5f6.pdf"
                  ↓
Download URL:     "http://.../download/a1b2c3d4e5f6"
                  ↓
Download Name:    "Annual_Report_2026.pdf" (original name restored)
```

[INFO] Users never see the UUID filename. Original name is restored during download via send_from_directory(download_name=...).

7.3 Download Process

- User clicks download link: /download/{file_id}
- Flask queries database for file_id
- Expiry check: if datetime.now() > expiry_time → delete & redirect
- If valid: increment download_count, update last_download_time
- Send file: send_from_directory(UPLOAD_FOLDER, stored_filename, download_name=original_name)
- Browser receives file with original filename

8. EMAIL NOTIFICATION SYSTEM

8.1 How Emails Are Sent

- Flask calls send_email_all_methods() function
- Method 1: PowerShell Integrated Auth (no credentials needed)
  - - Uses current Windows session credentials
  - - Runs: Send-MailMessage -SmtpServer EXCHANGE_SERVER
- Method 2: PowerShell with Explicit Credentials (fallback)
  - - Uses session-stored password: session['ad_password']
  - - Creates PSCredential object for authentication
- If both fail → error logged in database (email_error column)
- Sender acknowledgment email sent separately via send_sender_acknowledgment()

8.2 Email Template Structure

```
RECIPIENT EMAIL INCLUDES:
├── Sender info (name, email, domain)
├── File details (name, size, upload time)
├── Expiry warning (e.g., "Expires in 3 days")
├── Download button (green, prominent)
├── Direct link (text URL below button)
└── Optional message from sender (if provided)

SENDER ACKNOWLEDGMENT INCLUDES:
├── Success confirmation banner
├── File details summary
├── Recipient email address
├── Download link (for sender's reference)
└── Expiry timestamp
```

[INFO] All emails are HTML formatted with inline CSS for email client compatibility.

9. SECURITY IMPLEMENTATION

9.1 Security Features

Feature	Implementation
Authentication	Active Directory LDAP bind (5 format attempts)
Session Management	Flask secret_key + server-side sessions
File Upload	secure_filename() prevents path traversal
File Storage	UUID obfuscation hides original filenames
Access Control	AD group-based external email permissions
Admin Protection	@admin_required decorator checks ADMIN_USERS list
Audit Logging	IP address, user agent, timestamp for every action
Auto-Expiry	Files auto-delete after configured days

9.2 Known Security Issues & Fixes

Issue	Severity	Fix
Hardcoded AD_SERVICE_PASSWORD	CRITICAL	Move to environment variable: os.environ.get('AD_PASSWORD')
Hardcoded secret_key	CRITICAL	Generate random: os.urandom(24) or env variable
Password stored in session	HIGH	Remove session['ad_password']; re-authenticate for email
Debug mode enabled	MEDIUM	Set app.run(debug=False) in production

10. DEPLOYMENT GUIDE

10.1 Prerequisites

- Windows Server or Windows 10/11 PC
- Python 3.10+ installed
- Active Directory accessible (port 389/636 open)
- Exchange Server accessible (port 25 open)
- PowerShell available (for email sending)
- Static public IP (for external access)
- Sophos XG310 configured (port forwarding)

10.2 Installation Steps

```
# Step 1: Clone/Copy project to server
C:\> cd C:\inetpub\wwwroot\FileShare

# Step 2: Create virtual environment
C:\> python -m venv venv

# Step 3: Activate virtual environment
C:\> venv\Scripts\activate

# Step 4: Install dependencies
(venv) C:\> pip install -r requirements.txt

# Step 5: Verify installation
(venv) C:\> python -c "import flask; print(flask.__version__)"

# Step 6: Run application (development)
(venv) C:\> python app.py

# Step 7: Access in browser
http://localhost:5000
```

10.3 Production Deployment (IIS)

```
# Install IIS + CGI + FastCGI
# Install wfastcgi
(venv) C:\> pip install wfastcgi
(venv) C:\> wfastcgi-enable

# Configure web.config in project root:
<configuration>
  <system.webServer>
    <handlers>
```

```
      <add name="Python" path="*" verb="*"
           modules="FastCgiModule"

 scriptProcessor="C:\path\to\venv\Scripts\python.exe|C:\path\to\venv\Lib\site-
 packages\wfastcgi.py"
           resourceType="Unspecified"
           requireAccess="Script"/>
    </handlers>
  </system.webServer>
  <appSettings>
    <add key="PYTHONPATH" value="C:\path\to\FileShare" />
    <add key="WSGI_HANDLER" value="app.app" />
  </appSettings>
</configuration>
```

[INFO] For IIS deployment, use Windows Authentication (Kerberos) instead of manual login for seamless SSO experience.

11. TROUBLESHOOTING GUIDE

11.1 Common Issues & Solutions

Problem	Cause	Solution
AD login fails	Wrong credentials or AD server unreachable	Check AD_SERVER IP, verify firewall port 389/636 open
Email not sent	Exchange server down or auth failed	Check EXCHANGE_SERVER IP, test SMTP with Telnet
File upload fails	MAX_CONTENT_LENGTH exceeded	Check file < 2GB, or increase app.config['MAX_CONTENT_LENGTH']
External user can't download	BASE_URL is private IP	Change BASE_URL to public IP/domain in app.py
Database locked	Concurrent access to SQLite	Use connection pooling or switch to PostgreSQL/MySQL
Permission denied on uploads	IIS app pool identity	Grant IIS_IUSRS write permission on uploads/ folder
CSS not loading	Static files not served	Ensure static/ folder exists and IIS static file handler enabled

11.2 Debug Mode

```
# Enable detailed logging in app.py
import logging
logging.basicConfig(level=logging.DEBUG)

# Check console output for:
# [AD AUTH] Trying UPN format: user@barretthodgson.com
# [AD AUTH] SUCCESS with UPN format
# [AD AUTH] ALL FORMATS FAILED

# Check database directly:
(venv) C:\> python
>>> import sqlite3
>>> conn = sqlite3.connect('audit_logs.db')
>>> c = conn.cursor()
>>> c.execute("SELECT * FROM uploads ORDER BY upload_time DESC LIMIT 5")
>>> for row in c.fetchall(): print(row)
```

12. API REFERENCE

12.1 Public Routes

Route	Method	Auth Required	Description
/	GET	Yes (AD)	Main upload page
/login	GET/POST	No	AD authentication page
/logout	GET	No	Clear session and redirect
/upload	POST	Yes	Handle file upload + email
/download/<file_id>	GET	No	Download file by UUID
/api/user	GET	Yes	JSON: current user info
/api/check_rights	GET	Yes	JSON: external email permissions

12.2 Admin-Only Routes

Route	Method	Auth Required	Description
/admin/logs	GET	Admin only	Audit logs HTML page
/admin/api/logs	GET	Admin only	JSON: all upload records
/admin/api/stats	GET	Admin only	JSON: aggregate statistics

12.3 API Response Examples

```
# GET /api/user
{
  "username": "habiburrehman",
  "domain": "barretthodgson.com",
  "email": "habiburrehman@barretthodgson.com",
  "display_name": "Habib Ur Rehman",
  "can_send_external": true
}

# GET /admin/api/stats
{
  "total_files": 156,
  "total_size_mb": 2458.32,
  "total_downloads": 342
}

# GET /admin/api/logs
[
  {
    "id": 1,
```

```
    "file_id": "a1b2c3d4e5f6",
    "original_name": "report.pdf",
    "file_size": 5242880,
    "uploaded_by": "Habib Ur Rehman",
    "recipient_email": "user@example.com",
    "email_sent": true,
    "download_count": 3
  }
]
```

13. MAINTENANCE & UPDATES

13.1 Regular Maintenance Tasks

- Weekly: Check audit_logs.db size — archive old records if > 500MB
- Weekly: Review admin logs for failed login attempts
- Monthly: Verify AD service account password not expired
- Monthly: Test email delivery to external addresses
- Monthly: Check uploads/ folder size — cleanup if needed
- Quarterly: Update Python packages (pip install --upgrade)
- Quarterly: Review ADMIN_USERS list for ex-employees
- Annually: Security audit — check for hardcoded credentials

13.2 Backup Strategy

```
# Daily backup script (Windows Task Scheduler)
@echo off
set BACKUP_DIR=C:\Backups\FileShare
set DATE=%date:~-4,4%%date:~-10,2%%date:~-7,2%

# Backup database
xcopy /Y audit_logs.db %BACKUP_DIR%\audit_logs_%DATE%.db

# Backup uploads (incremental)
robocopy uploads %BACKUP_DIR%\uploads /MIR /FFT

# Keep last 30 days
cd %BACKUP_DIR%
forfiles /M "*.db" /D -30 /C "cmd /c del @path"
```

13.3 Scaling Considerations

- Current: SQLite (single file, good for < 1000 users)
- Scale up: PostgreSQL/MySQL for concurrent multi-user access
- Current: Single Flask instance (development server)
- Scale up: Gunicorn/uWSGI + NGINX reverse proxy
- Current: Local filesystem storage
- Scale up: Network-attached storage (NAS) or cloud blob storage
- Current: PowerShell email (single thread)
- Scale up: Celery + Redis for async email queue